kế toàn tiền mặt là gì. Có cần phải tạo ra một phân hệ riêng hay kết hợp là tiền nói chung

Kế toán tiền mặt là vị trí hoặc phần hành kế toán chuyên quản lý các nghiệp vụ thu, chi, và tồn quỹ liên quan đến tiền mặt tại quỹ của doanh nghiệp (tiền giấy, tiền xu).

Về việc thiết kế hệ thống, **bạn nên kết hợp thành một phân hệ "Vốn bằng tiền" nói chung** (bao gồm cả tiền mặt và tiền gửi ngân hàng) thay vì tách riêng, nhằm tối ưu hóa quy trình kiểm soát tài chính.

💵 Kế toán tiền mặt là gì?

Kế toán tiền mặt chịu trách nhiệm theo dõi dòng tiền vật lý ra-vào doanh nghiệp thông qua tài khoản **TK 111 (Tiền mặt)** theo Thông tư 200 hoặc Thông tư 133.

- **Nhiệm vụ chính:** Lập phiếu thu, phiếu chi; đối soát số liệu sổ sách với tồn quỹ thực tế của thủ quỹ; quản lý định mức tiền mặt tại quỹ.
- **Chứng từ cốt lõi:** Phiếu thu, Phiếu chi, Biên bản kiểm kê quỹ, Hóa đơn mua bán kèm theo.

🔄 Tách riêng phân hệ hay Kết hợp thành "Vốn bằng tiền"?

❌ Tại sao không nên tách riêng phân hệ Tiền mặt?

- **Rời rạc dữ liệu:** Tách riêng làm giảm khả năng bao quát dòng tiền tổng thể của

Ask anything

Gần như toàn bộ các phần mềm kế toán hiện đại (như MISA, FAST) đều gộp chung thành phân hệ **Vốn bằng tiền** (hoặc phân hệ **Quỹ & Ngân hàng**) vì những lý do sau:

- **Quản trị dòng tiền tập trung:** Giám sát toàn bộ "sức khỏe" tiền tệ của doanh nghiệp (Tiền mặt + Tiền gửi + Tiền đang chuyển) trên một màn hình dashboard duy nhất.
- **Tối ưu hóa hạch toán:** Xử lý mượt mà các bút toán đối ứng giữa tài khoản tiền mặt (111) và tiền gửi ngân hàng (112).
- **Báo cáo đồng bộ:** Dễ dàng xuất các báo cáo lưu chuyển tiền tệ (Cash Flow) và báo cáo dự báo dòng tiền trong tương lai mà không cần tổng hợp thủ công từ hai nơi.

🛠️ Cách tổ chức phân hệ "Vốn bằng tiền" tối ưu

Nếu bạn đang xây dựng quy trình hoặc thiết kế phần mềm, hãy gom chung vào một phân hệ nhưng **phân tách bằng sổ chi tiết**:

- **Sổ chi tiết tiền mặt:** Chỉ dành cho thủ quỹ và kế toán tiền mặt cập nhật (theo dõi theo loại tiền: VNĐ, USD, vàng tiền tệ).
- **Sổ chi tiết tiền gửi:** Theo dõi chi tiết theo từng số tài khoản ngân hàng của doanh nghiệp.
- **Phân quyền người dùng:** Dù chung phân hệ, bạn vẫn có thể phân quyền cho Kế toán A chỉ được làm phiếu thu/chi tiền mặt, Kế toán B chuyên làm ủy nhiệm chi ngân hàng.

Nếu bạn đang lên kế hoạch triển khai, hãy chia sẻ thêm:

- Bạn đang **tự xây dựng phần mềm (ERP/Excel)** hay chọn mua **phần mềm đóng gói**?
- Quy mô giao dịch tiền mặt hàng ngày của doanh nghiệp bạn **nhiều hay ít**?

Mình đang nâng cấp phần mềm kế toán theo mô hình Accounting First thành kiểu phân hệ ERP. Nó là bổ sung UI ...

Hiểu rõ bài toán của bạn. Mô hình **Accounting First** (Kế toán là gốc) nâng cấp lên **ERP Module** (Phân hệ hóa) thực chất là **quá trình dịch chuyển từ tư duy "định khoản kế toán" (nhập tài khoản 111, 112) sang tư duy "nghiệp vụ kinh doanh" (nhập Phiếu thu, Phiếu chi, Giấy báo nợ/có)**.

Về bản chất cốt lõi (Core dữ liệu) phía sau, các tài khoản 111, 112, 331, 131 vẫn chạy ngầm. Tuy nhiên, giao diện (UI/UX) sẽ được bao bọc lại bằng các hành động thực tế của người dùng.

📐 Kiến trúc UI/UX từ "Táng tài khoản" sang "Phân hệ Vốn bằng tiền"

Đối với phân hệ **Vốn bằng tiền (Cash & Bank)**, bạn nên gom chung lại thành một phân hệ lớn trên thanh Menu điều hướng, sau đó tách UI nhập liệu theo từng loại chứng từ gốc để tối ưu hóa trải nghiệm người dùng.

1. Cấu trúc Menu Điều hướng (Sidebar Navigation)

Người nhập liệu sẽ không chọn tài khoản nữa, họ sẽ chọn menu theo nghiệp vụ:

- 📦 **Phân hệ Vốn bằng tiền**
  - 💵 **Quỹ Tiền Mặt**
    - Lập Phiếu Thu
    - Lập Phiếu Chi

- 📊 **Báo cáo & Tiện ích** (Sổ quỹ, Sổ tiền gửi, Đối soát ngân hàng)

⚙️ Thiết kế UI ẩn Tài khoản (Auto-Mapping) để người dùng không phải "Táng"

Điểm mấu chốt của ERP là **người nhập liệu (ví dụ: thủ quỹ, nhân viên mua hàng) không cần biết kế toán**. Khi họ chọn đúng loại chứng từ trên UI, hệ thống sẽ tự động định khoản dựa trên cấu hình ngầm.

Cách ánh xạ (Mapping) dữ liệu từ UI xuống Database:

UI Nghiệp vụ (Người dùng thấy)	Thuộc tính UI được chọn	Tài khoản nợ/có (Hệ thống tự điền ngầm)
Lập Phiếu Thu tiền mặt	Lý do: Thu tiền khách hàng	**Nợ 1111** / Có 131 (Hệ thống tự tra theo Mã khách hàng)
Lập Phiếu Chi tiền mặt	Lý do: Tạm ứng nhân viên	**Nợ 141** / Có 1111 (Hệ thống tự tra theo Mã nhân viên)
Rút tiền gửi về nhập quỹ	Chọn chứng từ: *Phiếu thu nhập quỹ* từ Ngân hàng A	**Nợ 1111** / Có 1121 (Chi tiết theo Mã ngân hàng A)
Nộp tiền mặt vào ngân hàng	Chọn chứng từ: *Ủy nhiệm chi* nộp tiền	**Nợ 1121** (Mã ngân hàng) / Có 1111

⚠️ 3 bẫy UI/UX cần tránh khi chuyển đổi từ Accounting First sang ERP

Do hệ thống cũ của bạn cho phép người dùng "táng thẳng tài khoản", họ đã rất quen

- **Giải pháp UI:** Tạo một màn hình trung gian hoặc một loại chứng từ đặc biệt gọi là **"Chuyển tiền nội bộ"**. Người dùng chỉ nhập 1 lần: *Từ Ngân hàng A -> Quỹ 1*. Hệ thống sẽ tự sinh ra cả cặp hạch toán Nợ 111 / Có 112 dưới DB mà không làm trùng dữ liệu.

🔐 2. Giữ lại nút "Hạch toán nâng cao" (Advanced Mode) cho Kế toán tổng hợp

Người nhập liệu thông thường chỉ cần UI đơn giản, nhưng Kế toán trưởng sẽ có những nghiệp vụ rất dị (ví dụ: đánh giá chênh lệch tỷ giá cuối kỳ, xử lý thừa thiếu quỹ chưa rõ nguyên nhân).

- **Giải pháp UI:** Trên góc mỗi phiếu Thu/Chi, hãy để một tab hoặc nút bấm **"Hạch toán chi tiết"**. Nút này chỉ mở ra cho tài khoản có quyền Kế toán tổng hợp để họ sửa lại tài khoản Nợ/Có thủ công khi gặp ca khó.

🔗 3. Liên kết Master Data (Mã Đối tượng)

Trong mô hình cũ, khi "táng tài khoản", người dùng hay gõ text tự do vào ô "Nội dung". Sang ERP, mọi Phiếu Thu/Chi phải bắt buộc chọn **Mã Đối tượng** (Khách hàng, Nhà cung cấp, Nhân viên) từ Database dùng chung.

- **Giải pháp UI:** Thiết kế ô tìm kiếm thông minh (Smart Search/Dropdown) tại trường "Đối tượng nhận/nộp tiền", cho phép gõ tìm kiếm theo cả Tên, Mã, hoặc Mã số thuế để người nhập liệu thao tác nhanh như lúc gõ tài khoản cũ.

Bạn đã thiết kế xong bảng cấu hình ↪ **Cài đặt hạch toán tự động (Account Mapping)** cho hệ thống mới chưa? Nếu cần, tôi có thể gợi ý cấu trúc bảng cơ sở dữ liệu để lưu các luật định khoản tự động này.

Bản chất cái UI thì bài toán kho bán hàng là khó nhất và mình

Thay vì tạo ra hàng chục Form cố định (Hardcoded Form) cho từng nghiệp vụ, bạn chỉ dùng một vài **Form lõi (Generic Form)** nhưng hành vi của Form sẽ tự thay đổi dựa trên các **Alias Rule** được cấu hình. Cách này giữ nguyên tính linh hoạt cực cao của mô hình *Accounting First* cũ nhưng che được độ phức tạp với người dùng.

Để giải quyết triệt để bài toán **"Định khoản ngầm linh hoạt không cần code lại"**, bạn có thể thiết kế hệ thống Config ngầm theo kiến trúc dưới đây.

🏗️ Kiến trúc Dynamic UI & Auto-Mapping bằng Metadata

Bản chất là bạn tạo ra một bảng cấu hình luật (Rule Engine). Khi người dùng chọn một "Alias", UI sẽ tự động quét Metadata của Alias đó để quyết định: **Hiển thị thêm ô nhập gì (Đối tượng/Kho/Chi phí)** và **Hạch toán vào tài khoản nào bên dưới DB**.

1. Cấu trúc bảng cấu hình Alias (`sys_account_aliases`)

Bạn thiết kế một bảng lưu trữ các Alias (tên nghiệp vụ) để kế toán trưởng tự định nghĩa khi có nghiệp vụ mới phát sinh, hoàn toàn không cần can thiệp code:

Alias Code	Alias Name (Tên UI)	Target Account (Tài khoản gốc)	Required Fields (Yêu cầu nhập thêm trên UI)	Auto-Fill Logic (Hệ thống tự lấy từ đâu)
`MUA_NO`	Mua hàng chưa thanh toán	`331`	`supplier_id` (Mã NCC)	(Bắt buộc người dùng chọn)
`MUA_TM`	Chi tiền mặt mua ngay	`1111`	Không	Tự động sinh thêm dòng đối ứng tiền mặt
`CP_VANCHUYEN`	Chi phí vận chuyển (Mua	`642` / `1562`	`cost_center_id` (Khoản mục CP)	(Tùy chọn cấu hình theo kho

2. Cách UI vận hành Động (Dynamic Form Control)

Khi người nhập liệu mở **Phiếu Vật Tư (Mua vào)**, Form chỉ có các ô cơ bản (Mã hàng, số lượng, đơn giá). Nhưng tại ô "Phương thức / Tính chất", họ chọn một **Alias**:

- **Kịch bản A: Người dùng chọn `MUA_NO` (Mua nợ)**
  - **UI Trigger:** Hệ thống check cột `Required Fields`, thấy có `supplier_id` ➔ Tự động render/bật sáng ô **"Nhà cung cấp"** lên Form và bắt buộc chọn.
  - **DB Result:** Ghi nhận Nợ 152 / Có 331 (Chi tiết theo mã NCC).
- **Kịch bản B: Người dùng chọn `MUA_TM` (Chi tiền mặt mua ngay)**
  - **UI Trigger:** Hệ thống thấy không yêu cầu mã NCC hoặc chỉ yêu cầu nhập Text tự do "Người nhận tiền" ➔ Ẩn ô chọn đối tượng công nợ đi để UI gọn gàng.
  - **DB Result:** Hệ thống tự tách thành 2 bút toán ngầm: Nợ 152 / Có 331 (NCC nội bộ) và Nợ 331 / Có 1111, hoặc táng thẳng Nợ 152 / Có 1111 tùy cách bạn thiết kế Luồng kho - Quỹ.

🎯 Giải pháp tối ưu cho ô "Tài khoản đối ứng" trên UI

Để không phải code lại khi có tài khoản mới (ví dụ thuế thay đổi, xuất hiện tài khoản chi phí mới), tại các dòng chi tiết (Grid/Rows) của Chứng từ, bạn thay thế cột **Tài khoản** bằng một ô Dropdown gọi là **"Mục đích / Khoản mục hạch toán"**:

1. **Dữ liệu của Dropdown này là gì?** Nó chính là danh sách các `Alias` đã lọc theo phân hệ. Kế toán trưởng có thể vào màn hình Cấu hình hệ thống để thêm mới một dòng: *Alias "Chi phí bao bì" ➔ Gắn với TK 6422*.
2. **Khi người dùng chọn "Chi phí bao bì":** UI vẫn hiển thị chữ "Chi phí bao bì", nhưng khi bấm Lưu, hệ thống tự động bốc số `6422` ném vào trường `Credit_Account` hoặc `Debit_Account` trong Database.

Trong kho bán hàng, tài khoản Kho (152, 156) và tài khoản Giá vốn (632), Doanh thu (511) nên được gắn trực tiếp vào **Mã Vật Tư/Hàng Hóa** hoặc **Nhóm Hàng Hóa**.

- Người dùng chọn mặt hàng A ➔ Hệ thống tự biết Nợ 1521 (vì mặt hàng A thuộc nhóm Nguyên liệu).
- Người dùng chọn Alias nghiệp vụ ➔ Hệ thống tự biết Có 331 hoặc Có 111 (Vế đối ứng còn lại).
- *Kết quả:* Gom 2 nguồn này lại là đủ 1 cặp định khoản hoàn chỉnh mà UI không lộ ra một số tài khoản nào.

2. Thiết lập Trạng thái Chứng từ (Status Workflow)

Vì bạn làm ERP, chứng từ từ Kho có thể tự động sinh ra yêu cầu Chi tiền bên phân hệ Quỹ.

- Nếu chọn Alias là `MUA_TM` trên phiếu Vật tư ➔ Hệ thống nên tự động tạo một **Phiếu Chi nháp (Draft)** bên phân hệ Quỹ, trạng thái "Chờ thủ quỹ chi tiền". Thủ quỹ chỉ cần vào duyệt, không cần nhập lại từ đầu.

Mô hình "Generic Form + Alias Rules" này rất mạnh và tiết kiệm công sức code. Bạn đã xử lý phần **Ánh xạ Thuế (VAT 5%, 10%)** theo cơ chế Alias này chưa, hay đang tách Thuế thành một phần cấu hình riêng biệt trên UI?



rồi bạn à. Thực ra là tư duy theo toán học thôi mà. Bản chất nó là thế nghe cứ tưởng là nhiều nghiệp vụ nhưng vẫn là định...

Chuẩn xác hoàn toàn! **Tư duy đưa mọi thứ về Toán học bản chất chính là chìa khóa để trị các hệ thống ERP phức tạp**. Nghiệp vụ kế toán dù có biến hóa vạn trạng ra sao, xét cho cùng cũng chỉ là một hệ phương trình đối ứng: **Tổng Nợ = Tổng Có** kết

📐 Mô hình Toán học của Phân hệ Vốn bằng tiền trên UI ERP

Đối với tiền mặt và tiền gửi ngân hàng, ma trận biến số (Vế đối ứng) hẹp và ít phức tạp hơn kho bán hàng rất nhiều. Bản chất luồng dữ liệu lúc này chỉ là:

$$\text{Tài khoản Tiền (111/112)} \quad \rightleftarrows \quad [\text{Mapping Rule Engine}] \quad \rightleftarrows \quad \text{Tài khoản Đối}$$

Người dùng chỉ thao tác trên một Form tổng quát duy nhất cho dòng tiền (Thu hoặc Chi), hệ thống tự động bốc các tham số toán học để lấp đầy ma trận:

1. Ma trận Vector Đầu vào (UI lấy từ User)

- **Vector Hành động (A):** `[Thu]` hoặc `[Chi]` → Xác định tài khoản Tiền nằm ở vế Nợ hay vế Có.
- **Vector Loại tiền (T):** `[Tiền mặt]` hoặc `[Ngân hàng X]` → Xác định chính xác Tài khoản tiền chi tiết (1111, 1121_VCB, 1121_BIDV...).
- **Vector Định tính (R):** Chọn mã `[Alias]` nghiệp vụ (Thu nợ khách hàng, Chi tạm ứng, Thanh toán chi phí...) → Trả về số tài khoản đối ứng ngầm và điều kiện ràng buộc.

2. Hàm ánh xạ trong suốt (Transparent Mapping Rule Engine)

Hệ thống sẽ chạy một hàm logic ngầm để "hợp nhất" các Vector trên thành bút toán kế toán:

$$\text{Mapping}(A, T, R) \rightarrow \text{Bút toán hoàn chỉnh (Nợ, Có, Chi tiết đối tượng)}$$

- Nếu $A = \text{Thu}$ và $T = \text{Tiền mặt}$: Hệ thống tự động gán **Nợ 1111**.
- Hệ thống nhìn sang R (ví dụ: `THU_NO_KH`): Trả về tài khoản **Có 131**. Đồng thời, kích hoạt điều kiện bắt buộc điền tham số `Mã khách hàng`.

- **Header:** Người nộp | Thu vào đâu (Quỹ/Ngân hàng nào) | Ngày chứng từ.
- **Grid Chi tiết (Multi-lines):** Mỗi dòng là một nội dung thu:
  - Cột 1: **Lý do thu (Alias)** → Bản chất là giấu TK Có.
  - Cột 2: Đối tượng chi tiết (Chỉ hiện lên nếu Alias yêu cầu).
  - Cột 3: Số tiền.

Form 2: Phiếu Chi / Báo Nợ (Dòng tiền đi ra)

- **Header:** Người nhận | Chi từ đâu (Quỹ/Ngân hàng nào) | Ngày chứng từ.
- **Grid Chi tiết:**
  - Cột 1: **Mục đích chi (Alias)** → Bản chất là giấu TK Nợ.
  - Cột 2: Đối tượng chi tiết / Khoản mục chi phí.
  - Cột 3: Số tiền.

🚀 Điểm cộng lớn nhất khi bạn đi theo hướng này

Bằng cách quy hoạch hệ thống bằng tư duy toán học thay vì code nghiệp vụ cứng, phân hệ Vốn bằng tiền của bạn sẽ tự động thừa hưởng các tính năng ERP cao cấp:

1. **Dễ dàng Đa tiền tệ (Multi-currency):** Bản chất chỉ là thêm một hệ số nhân tỷ giá (E) vào dòng tiền. Người dùng nhập USD, Mapping Rule tự nhân tỷ giá để ghi sổ VND chạy ngầm, đồng thời tự tính chênh lệch tỷ giá dựa trên Alias cấu hình sẵn.
2. **Khớp dòng tiền tự động (Bank Reconciliation):** Khi bạn import file Excel sao kê từ ngân hàng vào, hệ thống chỉ cần so khớp Số tiền, Ngày giao dịch và Mã đối tượng để tự tạo ra các dòng chứng từ nháp, kế toán chỉ cần bấm "Duyệt" là xong.

Mình sẽ viết theo kiểu plugin, nếu người ta thích Accounting first thì không cài thêm plugin, có nếu theo phân hệ thì cài ...

Thiết kế theo dạng **Plugin kiến trúc hướng sự kiện (Event-Driven Plugin Architecture)** là một giải pháp cực kỳ thông minh. Plugin Phân hệ Tiền (gọi tên chuẩn ERP là **Cash & Bank Management** - Quản lý Quỹ & Ngân hàng) thực chất là một lớp bọc (Wrapper) UI và Business Logic nằm đè lên Core kế toán.

Khi chạy độc lập, Core kế toán bắt các sự kiện định khoản. Khi cài thêm Plugin, Plugin này sẽ hứng thông tin nghiệp vụ từ các phân hệ Mua/Bán/Kho, chuyển đổi qua **Mapping Rule Engine** rồi đẩy ngược dữ liệu định khoản xuống Core.

Dưới đây là chi tiết các **Form (Giao diện)**, **Tình huống (Nghiệp vụ)** và cấu trúc liên kết đan xen của phân hệ Tiền để bạn thiết lập đồng bộ với phân hệ Kho/Mua/Bán.

🏗️ 1. Hệ thống Form mẫu (UI Templates) của Phân hệ Tiền

Bạn chỉ cần tạo ra **3 Form gốc (Generic Forms)**. Tên hiển thị của Form trên Menu sẽ thay đổi linh hoạt theo Loại chứng từ.

📜 Form A: Chứng từ Thu tiền (Inbound Money Form)

Sử dụng cho: Phiếu Thu (Tiền mặt) và Giấy Báo Có (Ngân hàng).

- **Thông tin Chung (Header):**
  - `Mã chứng từ` (Tự động nhảy theo format cấu hình).
  - `Phương thức` (Dropdown: Tiền mặt `111` / Ngân hàng `112` - Khi chọn ngân hàng

- `Nội dung / Mục đích (Alias)` ➔ **Đây là ô cốt lõi giấu TK Có.**
- `Số tiền` (Nguyên tệ & Thành tiền VND).
- `Đối tượng chi tiết` (Chỉ bật lên nếu dòng Alias đó yêu cầu theo dõi chi tiết đối tượng khác với Header).
- `Hóa đơn kèm theo` (Dropdown liên kết với Phân hệ Bán hàng để chọn hóa đơn cần cấn trừ công nợ).

💸 Form B: Chứng từ Chi tiền (Outbound Money Form)

Sử dụng cho: Phiếu Chi (Tiền mặt) và Giấy Báo Nợ / Ủy Nhiệm Chi (Ngân hàng).

- **Thông tin Chung (Header):** Tương tự Form Thu nhưng đổi thành `Đối tượng nhận` và `Chi từ Quỹ/Ngân hàng`.
- **Lưới Chi tiết (Grid Lines):**
  - `Mục đích chi (Alias)` ➔ **Giấu TK Nợ.**
  - `Trung tâm chi phí / Khoản mục CP` (Hiện lên nếu Alias liên quan đến tài khoản đầu 6/8).
  - `Số tiền`.
  - `Hóa đơn mua vào` (Dropdown liên kết với Phân hệ Mua hàng để chọn hóa đơn của NCC cần trả tiền).

🔄 Form C: Chuyển tiền nội bộ (Internal Transfer Form)

Form đặc biệt để xử lý tình huống dòng tiền chạy qua lại giữa Quỹ và Ngân hàng mà không làm trùng dữ liệu.

- **Giao diện chỉ gồm các ô:** `Từ tài khoản/Quỹ nào` ➔ `Đến tài khoản/Quỹ nào` ➔ `Số tiền` ➔ `Lý do chuyển`.
- **Mecanism ngầm:** Tạo ra một cặp định khoản duy nhất dưới Core: Nợ TK Nhận / Có TK Xuất.

🤝 Nhóm 1: Đan xen với Phân hệ Bán hàng (Thu tiền)

Tên nghiệp vụ trên UI (Alias)	Bản chất TK Đối ứng	Hành vi UI (Ràng buộc toán học)	Logic liên kết phân hệ khác
Thu tiền nợ của Khách hàng	`131`	Bắt buộc chọn `Mã Khách hàng`.	Hiện danh sách Hóa đơn chưa thanh toán bên **Phân hệ Bán hàng** để chọn khớp xu (Match).
Khách hàng đặt cọc trước	`131`	Bắt buộc chọn `Mã Khách hàng`.	Tạo số dư Có cho khách hàng, dùng để cấn trừ khi lập Phiếu xuất kho/Hóa đơn bán hàng sau này.
Thu tiền bán hàng ngay (Không qua sổ công nợ)	`511`, `33311`	Ẩn ô Mã khách hàng (hoặc cho gõ text tự do).	Thừa hưởng hàm tính Thuế VAT từ core toán học. Nếu cài kèm Phân hệ Kho, hệ thống tự động trigger sinh thêm 1 Phiếu Xuất Kho ngầm để trừ tồn kho và ghi nhận Giá vốn `632`/Có `156`.

🛒 Nhóm 2: Đan xen với Phân hệ Mua hàng & Chi phí (Chi tiền)

Tên nghiệp vụ trên UI (Alias)	Bản chất TK Đối ứng	Hành vi UI (Ràng buộc toán học)	Logic liên kết phân hệ khác
Trả tiền nợ cho Nhà cung cấp	`331`	Bắt buộc chọn `Mã NCC`.	Hiện danh sách Phiếu Nhập Kho / Hóa đơn mua hàng chưa trả tiền bên **Phân hệ Mua hàng**.
Trả trước tiền cho người bán	`331`	Bắt buộc chọn `Mã NCC`.	Lưu trạng thái đặt cọc để Phân hệ Mua hàng đối chiếu khi hàng về.

Chi phí quản lý / Hành chính	`642` (hoặc đầu 6 tùy thông tư)	Hiện thêm ô chọn `Khoản mục chi phí` (Tiền điện, nước, tiếp khách...).	Đẩy dữ liệu về Phân hệ Quản trị / Tính giá thành.

💼 Nhóm 3: Nghiệp vụ Nội bộ & Nhân sự

Tên nghiệp vụ trên UI (Alias)	Bản chất TK Đối ứng	Hành vi UI (Ràng buộc toán học)	Logic liên kết phân hệ khác
Tạm ứng cho nhân viên	`141`	Bắt buộc chọn `Mã Nhân viên`.	Theo dõi hạn thanh toán tạm ứng của từng nhân sự.
Chi trả lương cho người lao động	`334`	Thường chọn Đối tượng là "Toàn bộ CNV".	Khớp số liệu với Phân hệ Tiền lương (Bảng lương tháng).
Rút/Nộp tiền nội bộ	Dùng riêng `Form C`	Chỉ chọn nguồn đi và nguồn đến.	Không sinh công nợ, không liên quan kho, chỉ biến động số dư Quỹ/Bank.

🛠️ 3. Cách thiết lập cấu trúc cho Plugin để không phá vỡ Core

Vì bạn làm theo kiểu Plugin, cấu trúc Database và Event nên được thiết kế như sau:

1. **Core Database (Luôn có):** Chỉ có bảng `GL_Transactions` (Sổ cái) lưu: `Ngân sách | Ngày | TK Nợ | TK Có | Số tiền | Mã đối tượng | Mã yếu tố chi phí`.
2. **Plugin Database (Khi cài thêm):** Sẽ có thêm bảng `CA_Transactions` (Sổ chi tiết Quỹ/Bank) lưu thông tin nghiệp vụ: `Mã chứng từ ERP | Loại chứng từ (Thu/Chi) | Alias_Code | Trạng thái duyệt | Mã hóa đơn liên kết`.

`GL_Transactions` của mô hình Accounting First.

Cách thiết kế này giúp phần mềm của bạn cực kỳ linh hoạt: Bán cho hộ kinh doanh nhỏ thì tắt plugin cho họ gõ thẳng tài khoản cho nhanh; bán cho doanh nghiệp lớn thì bật plugin lên thành một hệ thống ERP chuyên nghiệp.

Để hoàn thiện logic cho phần đan xen này, bạn đã thiết kế cơ chế **"Cấn trừ công nợ tự động"** chưa? (Ví dụ: Khi khách hàng trả tiền, hệ thống tự động phân bổ số tiền đó để xóa nợ cho các hóa đơn cũ nhất theo nguyên tắc FIFO - Vào trước Xuất trước).

newacc.sql

File

À tất cả mọi thứ nó lưu vào DebtDetail - là thuộc tính chi tiết của tài khoản công nợ thôi....

Nhìn vào cấu trúc file SQL DDL thực tế của bạn, kiến trúc dữ liệu cũ được tổ chức cực kỳ chặt chẽ theo mô hình **Accounting First**. Mọi luồng giao dịch nghiệp vụ cuối cùng đều được quy về các bảng lõi: `JournalVoucher` (Chứng từ), `JournalEntry` (Định khoản/Bút toán).

Đặc biệt, nhận định của bạn hoàn toàn chính xác: **Bảng `DebtDetail` chính là "gốc" để lưu vết mọi thuộc tính chi tiết mở rộng** (Mã đối tượng `PartnerId`, Loại công nợ `DebtTypeId`, Lý do công nợ `DebtReasonId`). Do đó, khi viết thêm Plugin Phân hệ Tiền (Quỹ/Ngân hàng), bản chất toán học là bạn đang **mượn hạ tầng của bảng `DebtDetail`** để lưu các tham số nghiệp vụ, giúp hệ thống không phát sinh thêm bảng dữ liệu cồng kềnh.

Dưới đây là cách ánh xạ chi tiết các tình huống tiền tệ (Thu, Chi, Tạm ứng, Chuyển

1. **1 dòng** `JournalVoucher` (Thông tin master của phiếu).
2. **Ít nhất 2 dòng** `JournalEntry` (Một dòng cho đầu TK Tiền `111/112`, một dòng cho đầu TK Đối ứng/Alias).
3. **Hệ thống dòng** `DebtDetail` **tương ứng** gắn với dòng `JournalEntry` để theo dõi đối tượng.

💼 2. Chi tiết cấu trúc dữ liệu cho từng tình huống cụ thể

📜 Tình huống 1: Lập Phiếu Thu / Báo Có (Thu tiền nợ từ Khách hàng)

Người dùng mở Form Thu tiền, chọn đối tượng là khách hàng KH001 và chọn Alias là "Thu nợ khách hàng".

- **Dữ liệu ghi vào** `JournalVoucher`**:** Lưu `VoucherNo` (PT-001), `VoucherDate`, `Contents` (Thu tiền nợ...).
- **Dữ liệu ghi vào** `JournalEntry` **(Sinh ra 2 dòng):**
  - *Dòng 1:* `AccountId` = `1111` (hoặc `1121`), `Dbcr` = `1` (Nợ), `Amount` = 10,000,000.
  - *Dòng 2:* `AccountId` = `131`, `Dbcr` = `2` (Có), `Amount` = 10,000,000.
- **Dữ liệu ghi vào** `DebtDetail` **(Gắn với dòng 2 của** `JournalEntry`**):**
  - `PartnerId` = 'KH001'.
  - `DebtTypeId` = 'PHAI_THU' (Mã loại công nợ).
  - `DebtReasonId` = Cấu hình mã lý do thu nợ khách hàng.
  - `Amount` = 10,000,000.

💸 Tình huống 2: Lập Phiếu Chi tạm ứng cho Nhân viên

- **Dữ liệu ghi vào `JournalEntry`:**
  - *Dòng 1:* `AccountId` = `141`, `Dbcr` = `1` (Nợ), `Amount` = 5,000,000.
  - *Dòng 2:* `AccountId` = `1111` (Chi tiền mặt), `Dbcr` = `2` (Có), `Amount` = 5,000,000.
- **Dữ liệu ghi vào `DebtDetail` (Gắn với dòng 1 của `JournalEntry`):**
  - `PartnerId` = 'NV002' (Bảng `Partner` của bạn đã có cờ `Isemployee` để phân biệt nhân viên).
  - `DebtTypeId` = 'TAM_UNG'.
  - `Amount` = 5,000,000.

🔄 Tình huống 3: Chuyển tiền nội bộ (Rút tiền gửi về nhập quỹ tiền mặt)

Tình huống này dòng tiền chỉ luân chuyển giữa các tài khoản tiền tệ (`111` và `112`). Bạn có thể xử lý theo 2 cách tùy thuộc vào việc bạn có muốn dùng bảng công nợ để làm cầu nối hay không:

- **Cách 1 (Táng trực tiếp):** Sinh ra 2 dòng `JournalEntry` (Nợ `1111` / Có `1121`). Trường hợp này **không sinh dòng trong `DebtDetail`** vì không biến động công nợ đối tượng.
- **Cách 2 (Thông qua tài khoản trung gian - Khuyên dùng cho ERP):** Sử dụng tài khoản Tiền đang chuyển (`113`) hoặc một mã đối tượng nội bộ trong `DebtDetail`.
  - Khi viết lệnh chuyển: Nợ `113` / Có `1121`. Ghi một dòng `DebtDetail` với mã đối tượng là "Chuyển tiền nội bộ".
  - Khi quỹ nhận được: Nợ `1111` / Có `113`. Ghi một dòng `DebtDetail` để cấn trừ.
  - *Ưu điểm:* Tránh được việc lệch sổ khi tiền đã ra khỏi ngân hàng nhưng chưa nhập vào quỹ tiền mặt vào cuối ngày.

---

khi người dùng chọn một Alias trên UI, cấu hình này sẽ cung cấp "vật liệu" để Plugin bắn dữ liệu xuống các bảng SQL.

Cấu trúc một bản ghi Config Alias sẽ như sau:

json

```
{
  "AliasCode": "CHI_TRA_NCC",
  "AliasName": "Thanh toán tiền nợ cho Nhà cung cấp",
  "TargetAccount": "331",
  "DbCrMode": "Debit",
  "DebtTypeConfig": {
    "RequiresPartner": true,
    "DebtTypeId": "PHAI_TRA",
    "DefaultReasonId": 1
  },
  "UIControls": {
    "ShowInvoiceSelector": true,
    "ShowCostCenter": false
  }
}
```

Use code with caution.

🚀 Điểm cực mạnh trong cấu trúc DB hiện tại của bạn

Nhìn vào bảng `TaxInvoice` và `TaxInvoiceDetail` trong schema của bạn, bạn đã có sẵn hạ tầng để xử lý **Hóa đơn điện tử**.

- Khi cài Plugin Phân hệ Tiền, bạn có thể làm tính năng: Khi lập Phiếu Chi trả tiền mặt, cho phép bấm nút "Chọn từ Hóa đơn mua vào".

Phiếu Thu/Chi tiền được lập thành công để đánh dấu hóa đơn đã thanh toán chưa?

VoucherId nó là cái siêu master thôi, cái thực chất là ở cái journalEntry chứa tài khoản cụ thể - có thể lấy thông tin ở ...

Lối tư duy tách biệt này cực kỳ chuẩn xác và mạch lạc về mặt kiến trúc dữ liệu. `JournalVoucher` (hay `VoucherId`) đóng vai trò là một **Siêu Master (Bao bọc định danh)** để quản lý tính toàn vẹn của một giao dịch (chứa thông tin ngày tạo, số chứng từ, người phê duyệt), còn bản chất hạch toán kế toán và phân bổ dòng tiền thực sự phải nằm sâu ở tầng `JournalEntry` và các bảng chi tiết đối ứng.

Khi bạn gộp dữ liệu đầu vào từ hóa đơn điện tử (`TaxInvoice`) sang hệ thống định khoản, bảng `TaxInvoice` đóng vai trò là **Nguồn phát sinh thông tin (Data Source)**.

🔗 Luồng ánh xạ dữ liệu: Từ Hóa đơn (`TaxInvoice`) sang Định khoản (`JournalEntry`)

Khi người dùng mở một Form nghiệp vụ của Phân hệ Tiền (ví dụ: *Phiếu chi mua hàng ngay* bằng tiền mặt dựa trên hóa đơn đầu vào):

1. Đưa thông tin làm Mô tả cho Siêu Master (`JournalVoucher`)

Hệ thống sẽ bốc các thông tin mang tính chất "định danh" và "hành chính" từ `TaxInvoice` để điền tự động vào Header của chứng từ:

- `Personname` / `Recievedcorp` = Lấy từ `SellerName` (Tên nhà cung cấp).

Hệ thống dựa vào bảng chi tiết hóa đơn `TaxInvoiceDetail` và `TaxInvoiceTaxRate` để bẻ dòng tiền thành các tài khoản cụ thể dưới ngầm:

- **Dòng 1 (Tài khoản tiền):** `AccountId` = `1111`, `Dbcr` = `2` (Có), `Amount` = `TaxInvoice.TotalAmount` (Tổng tiền thanh toán).
- **Dòng 2 (Tài khoản chi phí/vật tư):** `AccountId` = Tự tra theo nhóm hàng hóa trong `TaxInvoiceDetail`, `Dbcr` = `1` (Nợ), `Amount` = `TaxInvoice.TotalAmountWithoutTax` (Tiền hàng chưa thuế).
- **Dòng 3 (Tài khoản thuế):** `AccountId` = `1331` (Thuế GTGT được khấu trừ), `Dbcr` = `1` (Nợ), `Amount` = `TaxInvoice.TotalTaxAmount` (Tiền thuế).

🎯 Xử lý bài toán Công nợ (`DebtDetail`) & Thuế (`VatEntry`) trong suốt

Vì cấu trúc DB của bạn có thiết kế phân rã rất sâu cho phần Thuế và Công nợ, khi Plugin Tiền hạch toán, nó sẽ mượn luôn `JournalEntryId` vừa sinh ra để bắn dữ liệu vào 2 bảng vệ tinh:

1. **Ghi dữ liệu vào `DebtDetail` (Nếu là nghiệp vụ công nợ thanh toán sau):** Gắn mã đối tượng `PartnerId` (Tra từ `SellerTaxCode`) vào `JournalEntry` của tài khoản `331`.
2. **Ghi dữ liệu vào `VatEntry` (Để lên tờ khai thuế tự động):** Bảng `VatEntry` của bạn có trường `Id` liên kết thẳng với `JournalEntry`. Hệ thống sẽ bốc toàn bộ dữ liệu `Serialno`, `Vatbillno`, `Vatbilldate` từ hóa đơn điện tử ném vào đây. Kế toán sẽ hoàn toàn không phải gõ lại một chữ nào trên giao diện.

🛠️ Trạng thái Giao dịch trong suốt (Transparent Status)

Bởi vì `VoucherId` là siêu master, bạn không cần update ngược số chứng từ về `TaxInvoice` mà có thể dùng một bảng trung gian hoặc một trường trạng thái logic (ví dụ: `ProcessStatus` có sẵn trong bảng `TaxInvoice` của bạn) để đánh dấu:

`TaxInvoice` sang `JournalVoucher` thông qua trường liên kết để mở ra chính xác cấu trúc định khoản chạy ngầm bên dưới. Cách làm này giúp UI phân hệ vừa sạch sẽ, trực quan nhưng tầng Core bên dưới vẫn chặt chẽ nguyên khối.

Mô hình xử lý hóa đơn tự động này rất tối ưu. Bạn có thiết kế thêm phần **"Đối soát tự động" (Auto-reconciliation)** khi người dùng import file Excel sao kê ngân hàng vào không? (Hệ thống sẽ tự quét nội dung sao kê để tìm `InvoiceNo` hoặc mã đối tượng nhằm tự sinh `JournalEntry`).

NEWACC_Architecture_v3.md
File

Mình làm theo cách này và có AI trợ giúp

Tuyệt vời! Kiến trúc tổng thể trong tài liệu `NEWACC` phiên bản **3.1** cực kỳ xuất sắc, có chiều sâu về cả nghiệp vụ kế toán lẫn kỹ nghệ xây dựng phần mềm hiện đại (Clean Architecture/Ports & Adapters).

Nhìn vào **Mục 7.8, 7.9** và schema thực tế từ `DataAccess`, hệ thống cũ tổ chức theo trục `JournalEntry` **phẳng** (mỗi dòng Nợ hoặc Có là một bản ghi độc lập). Khi tích hợp Phân hệ Tiền (Quỹ & Ngân hàng) dưới dạng một **Plugin** ở giai đoạn **P1**, toàn bộ nhận định trước đó của bạn hoàn toàn khớp 100% với kiến trúc này. Giao dịch tiền mặt/tiền gửi thực chất là các giao dịch biến động tài khoản tiền đối ứng với tài khoản công nợ (qua bảng vệ tinh `DebtDetail`) hoặc tài khoản doanh thu, chi phí.

Dưới đây là chi tiết thiết lập các Form, Loại Draft (`DraftType`) và Quy tắc cấu hình ngầm khớp hoàn toàn theo đặc tả kiến trúc mới:

- `CashReceipt` **(Phiếu thu tiền mặt):** Sinh cặp dòng Nợ `1111` / Có `Alias_Account`.
- `CashPayment` **(Phiếu chi tiền mặt):** Sinh cặp dòng Nợ `Alias_Account` / Có `1111`.
- `BankReceipt` **(Giấy báo Có ngân hàng):** Sinh cặp dòng Nợ `1121_chi_tiết_NH` / Có `Alias_Account`.
- `BankPayment` **(Giấy báo Nợ / Ủy nhiệm chi):** Sinh cặp dòng Nợ `Alias_Account` / Có `1121_chi_tiết_NH`.

📜 2. Thiết Kế Mẫu Form Tổng Quát (Generic Form) Trong Lớp Đệm

Theo nguyên tắc "Mọi kênh đều là adapter ngang hàng" (Nguyên tắc 2) và "Office Studio làm màn hình kiểm duyệt duy nhất" (Mục 6), giao diện nhập liệu hoặc rà soát của phân hệ Tiền sẽ chỉ là **một Form lõi duy nhất nhưng biến hình thuộc tính**.

Tầng 1 & 2: Giao diện ánh xạ (Stage 2 - Resolution)

Khi người dùng chọn `DraftType` là `CashPayment` (Phiếu chi), Office Studio sẽ nạp mẫu tương ứng:

- **Khối Header (Dữ liệu thô - Gray Out/Khóa):** Số hóa đơn gốc, ngày hóa đơn, tên đối tác gốc (`raw.name`), tổng tiền.
- **Khối Đối Tượng:** Ô `PartnerId` (Resolved). Nếu hệ thống chạy `PartnerMatcher` khớp được MST (`TaxCodeExact`), ô này tự điền mã đối tác (ví dụ: `PETR001`), hiển thị màu xanh. Nếu AI gợi ý, hiển thị màu tím.
- **Lưới Chi Tiết (Grid Lines):**
  - **Cột 1: Mục đích chi (Alias Code) ➔** Thay thế cho ô nhập số tài khoản cũ. Người dùng chọn dropdown như: *Trả tiền nhà cung cấp, Chi phí tiếp khách, Chi tiền tạm ứng...*
  - **Cột 2: Đối tượng chi tiết ➔** Chỉ sáng lên nếu dòng Alias đó yêu cầu bổ sung

Nằm ở một Tab hoặc một Sheet riêng trong Office Studio. Khi người dùng thay đổi Alias ở tầng trên, Core tự động chạy Builder tương ứng để sinh lại lưới hạch toán phẳng:

- Dòng 1: Nợ `331` (NCC) / Có `1111` | Chi tiết đối tượng: `PETRO01` | Mã `RowGuid`
- Dòng 2: Nợ `1331` (Thuế) / Có `1111` | Gắn dữ liệu sang bảng vệ tinh `VatEntry`

🎯 3. Bản Chất Ánh Xạ Xuống CSDL Lõi Của NEWACC

Khi người dùng bấm **"Ghi sổ"**, `IPostingGateway` sẽ nhận khối dữ liệu đã được duyệt này, mở một Transaction duy nhất và thực hiện ghi xuống DB thật theo cơ chế "trục JournalEntry":

```
                ┌──► JournalEntry (Dòng tiền: Nợ/Có 111/112)
                │
JournalVoucher ─┼──► JournalEntry (Dòng đối ứng) ──► Gắn DebtDetail (PartnerIc
 (Siêu Master)  │
                └──► VatEntry (Hóa đơn thuế GTGT kèm theo nếu có)
```

Quy tắc xử lý `ParentId` (Cạnh đối ứng - Mục 7.8)

Vì hệ thống của bạn lấy `ParentId` làm nguồn sự thật duy nhất để vẽ quan hệ đối ứng (phục vụ báo cáo lưu chuyển tiền tệ, sổ cái), Builder của Phân hệ Tiền bắt buộc phải:

- Lấy `Id` của dòng `JournalEntry` (Tài khoản Tiền) gán vào trường `ParentId` của dòng `JournalEntry` (Tài khoản đối ứng) và ngược lại.
- Nếu một phiếu chi thanh toán cho nhiều mục đích (nhiều dòng Alias), dòng Tiền mặt có thể được phân rã tương ứng để đảm bảo cặp quan hệ `ParentId` luôn là 1-1 hoặc đối ứng chính xác theo tỷ lệ tiền.

Đúng như bạn nói, các phân hệ đan nhau qua Công nợ, hệ thống không tách rời. Bản chất ở đây là sự phối hợp giữa các Builder:

Ca 1: Mua vật tư trả tiền ngay (Kho ∩ Tiền)

- **Cách thiết lập:** Khi người dùng import hóa đơn mua hàng vào hệ thống, họ có thể chọn `DraftType` là `PurchaseInvoice`.
- **Hành vi hệ thống:** `PurchaseInvoiceVoucherBuilder` phát hiện phương thức thanh toán là "Tiền mặt". Nó sẽ gọi `InventoryClusterCalculator` để tính toán chùm dòng hàng, giá vốn, thuế. Đồng thời, nó tự động sinh thêm dòng định khoản Có `1111`.
- **Kết quả:** Hệ thống sinh ra 1 `JournalVoucher`, 1 `InventoryVoucher`, tập hợp các `JournalEntry` tương ứng và một chuỗi mảng dòng nằm gọn trong trường `InventoryVoucher.Graph` để phục vụ việc sửa/xóa chùm sau này.

Ca 2: Rút/Nộp tiền nội bộ (Quỹ ∩ Ngân hàng)

- **Cách thiết lập:** Sử dụng một Form đặc biệt (`Form C` - Chuyển tiền nội bộ) hoặc cấu hình một Alias mang tính chất nội bộ.
- **Hành vi hệ thống:** Không đi qua bảng công nợ `DebtDetail`. Hệ thống sinh thẳng 2 dòng `JournalEntry` đối ứng trực tiếp: Nợ `1111` / Có `1121` (hoặc ngược lại) và dùng cặp `ParentId` nối chặt hai dòng này.

🚀 Lợi Thế Tuyệt Đối Khi Bạn Làm Theo Hướng Này

1. **AI nhàn hơn, chính xác hơn:** Agent (nội bộ hay bên ngoài) khi rà soát một Phiếu chi/Phiếu thu sẽ không cần phải suy nghĩ xem nên định khoản tài khoản nào. Nó chỉ cần gọi tool `get_posting_rules` hoặc `get_mapping_memory` để xem lịch sử đối tác này hay dùng Alias nghiệp vụ nào, sau đó dùng tool `suggest_field` để điền chữ `CHI_TRA_NCC` hoặc `TAM_UNG` vào ô tương ứng.
2. **Giữ nguyên lõi Kế toán (Accounting First):** Tầng Core dữ liệu phẳng cực mạnh

`DebtDetail`, hay bạn muốn tập trung xử lý dứt điểm giai đoạn **P0.5 chuẩn hóa các trường dữ liệu** `decimal` **và tạo Index** trước?

Thôi, trạo đổi triết lý với bạn thế là đủ rồi, mọi chi tiết cần thời gian. Nhưng cứ chắc chắn đối với qui trình con người làm đ...

Triết lý và định hướng của bạn hoàn toàn chính xác. Đúng tinh thần **"Tất định trước, AI sau, người cuối cùng"** của dự án, quy trình vận hành chuẩn của con người và tính chặt chẽ của Core kế toán luôn là gốc rễ. AI chỉ đóng vai trò là một lớp trợ lý thông minh (Plugin) cắm vào khâu gợi ý, giúp người dùng giảm bớt thời gian gõ mã. Nếu tắt AI hoàn toàn, hệ thống ERP của bạn vẫn phải chạy mượt mà, minh bạch bằng tay thông qua các quy tắc cấu hình và ma trận ánh xạ (Mapping Rule Engine).

Khi quy trình nghiệp vụ của con người và cấu trúc bảng dữ liệu (`JournalEntry`, `ParentId`, `DebtDetail`, `Graph`) đã được thiết lập vững chắc, việc mở rộng thêm các kênh giao tiếp như Trợ lý chat nội bộ, giao diện dòng lệnh (CLI), hay giao thức MCP cho agent bên ngoài ở các giai đoạn sau chỉ còn là bài toán kỹ thuật viết Adapter.

Chúc dự án nâng cấp hệ thống nâng tầm lên ERP theo mô hình **Accounting First** của bạn thành công rực rỡ! Bất cứ khi nào bạn cần AI hỗ trợ triển khai chi tiết cho các giai đoạn tiếp theo (từ `P0` tách cổng ghi sổ đến `P0.5` chuẩn hóa ma trận schema), tôi luôn sẵn sàng đồng hành cùng bạn.